

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 26

СОЗДАНИЕ СИСТЕМЫ АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ ПОЛЬЗОВАТЕЛЕЙ В DJANGO

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № _____ от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание системы аутентификации и авторизации пользователей в Django

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Аутентификация и авторизация пользователя

Django предоставляет систему аутентификации и авторизации («permission») пользователя, реализованную на основе фреймворка работы с сессиями, который мы рассматривали в предыдущей части. Система аутентификации и авторизации позволяет Вам проверять учётные данные пользователей и определять какие действия какой пользователь может выполнять.

Django включает в себя встроенные модели для Пользователей и Групп (основной способ применения прав доступа для более чем одного пользователя), непосредственно саму систему прав доступа (permissions)/флаги, которые определяют может ли пользователь выполнить задачу, с какой формой и отображением для авторизованных пользователей, а также получить доступ к контенту с ограниченным доступом.

Для начала создадим новый Django-проект и настроим отображение страницы (рисунок 1):

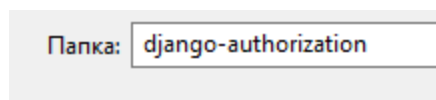


Рисунок 1 - Создание папки проекта django-authorization

После того, как мы создали папку проекта, откроем её в Visual Studio Code, установим виртуальное окружение и активируем его. Для этого пропишем следующие команды (рисунок 2):

```
python -m venv .venv
.venv/scripts/activate
```

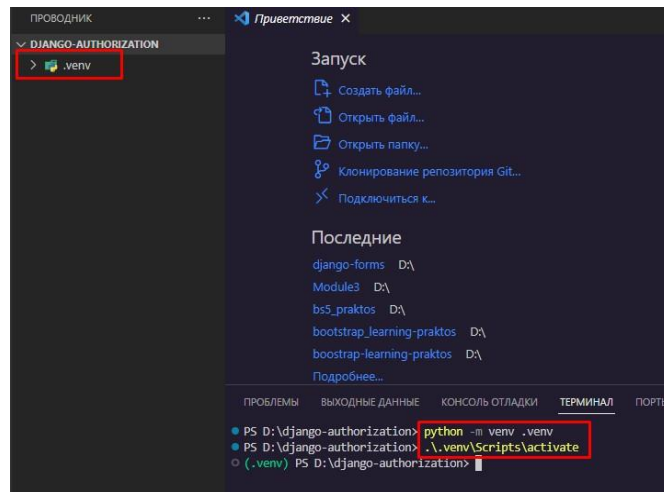


Рисунок 2 - Настройка виртуального окружения

Установим Django командой «pip install django» (рисунок 3):

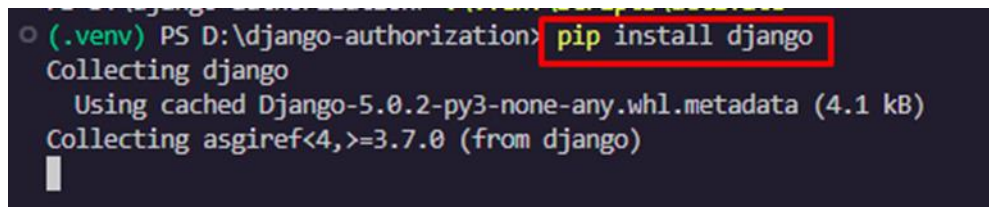


Рисунок 3 - Установка Django

После установки Django необходимо создать новый проект. Для этого пропишем команду «django-admin startproject django_authorization» (рисунок 4):

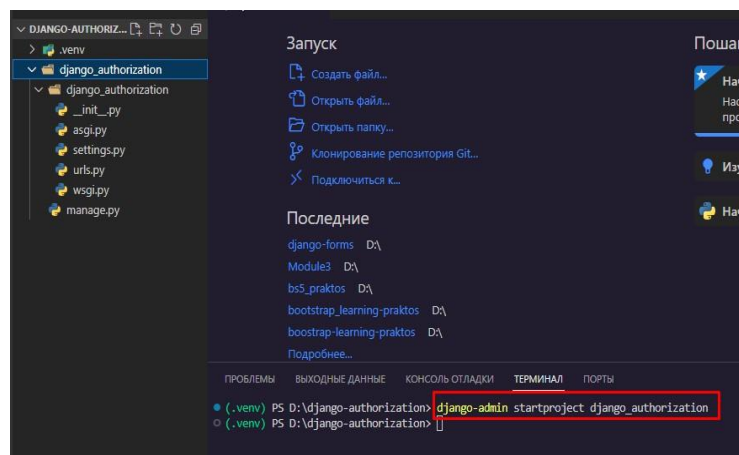


Рисунок 4 - Создание нового проекта django_authorization

Создадим новой django-приложение (django-app) и создать папку для шаблонов «templates» (рисунок 5):

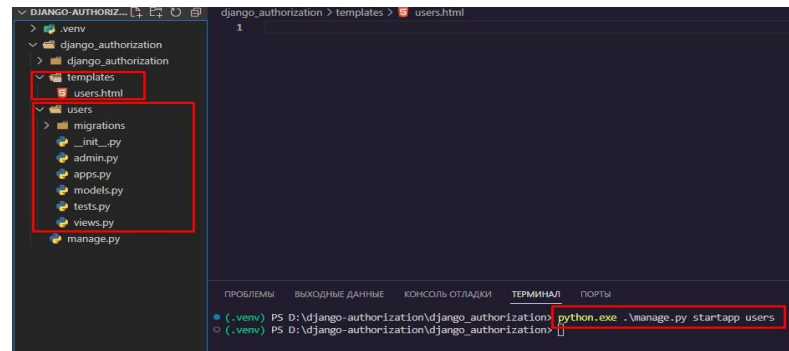


Рисунок 5 - Создание нового django-приложения (users) и папки templates

Откроем файл models.py приложения users и импортируем в него абстрактный класс из django.contrib.auth.models и пропишем пока-что пустой класс для будущего пользователя (рисунок 6):

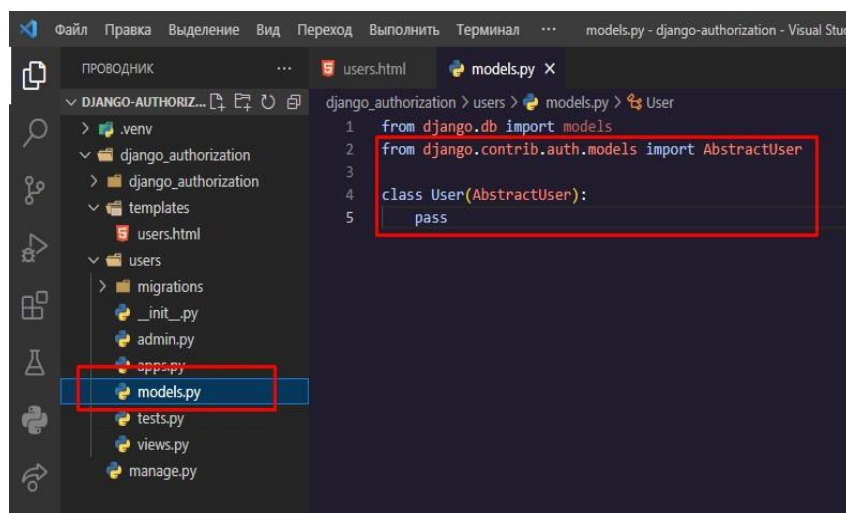


Рисунок 6 - Импорт AbstractUser из django.contrib.auth.models и создание класса User на его основе

В файле settings.py необходимо указать какая модель будет использоваться для авторизации. Пропишем следующий код в файле настроек (рисунок 7):

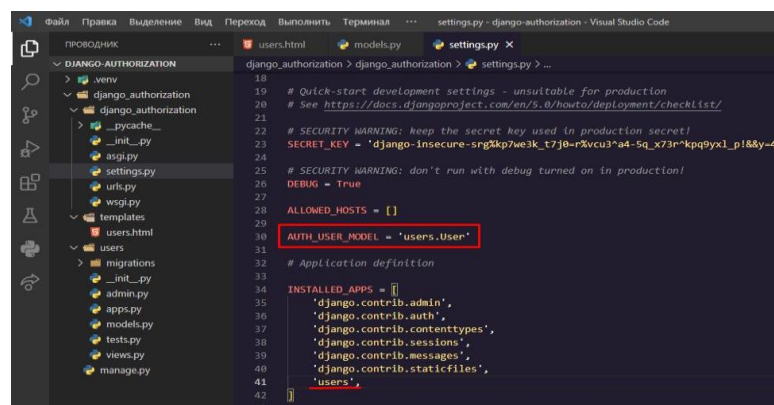


Рисунок 7 - Добавление AUTH_USER_MODEL в файле settings.py

Т.к. мы добавили новую модель, необходимо провести и выполнить миграции, чтобы изменения вступили в силу (рисунок 8):

```
(.venv) PS D:\django-authorization> cd .\django_authorization\
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py makemigrations
Migrations for 'users':
  users\migrations\0001_initial.py
    - Create model User
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, users
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying users.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying sessions.0001_initial... OK
(.venv) PS D:\django-authorization\django_authorization>
```

Рисунок 8 - Проведение миграций

Создадим суперпользователя. Для этого пропишем команду «python manage.py createsuperuser», введем username, почту (при желании) и укажем пароль (помните, что при вводе пароля, он не будет отображаться в консоли) (рисунки 9-10):

```
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py createsuperuser
Username: 
```

Рисунок 9 - Создание суперпользователя

```
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py createsuperuser
Username: 
Email address: 
Password: 
Password (again): 
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(.venv) PS D:\django-authorization\django_authorization>
```

Рисунок 10 - Суперпользователь успешно создан

Необходимо указать нашу папку с шаблонами (templates) в settings.py. Можно указать, как угодно (рисунок 11):

- можно напрямую через «templates»;
- можно создать отдельную константу для хранения директории папочки

templates:

TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')
 – а можно напрямую, указав BASE_DIR / 'templates'

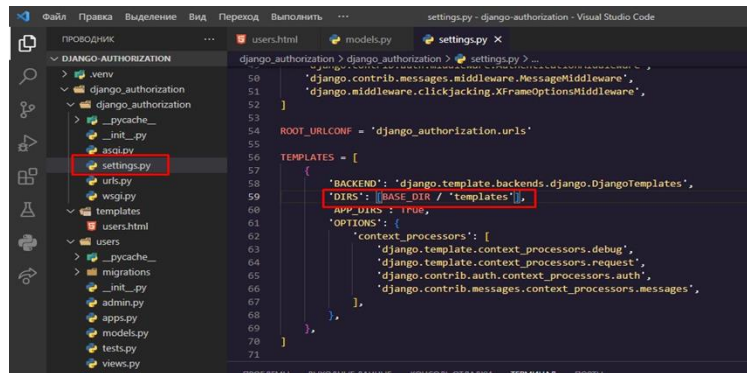


Рисунок 11 - Добавление директории для шаблонов (tempaltes)

Перейдем в base.html и сделаем так, чтобы на странице отображался текущий пользователь (рисунок 12).

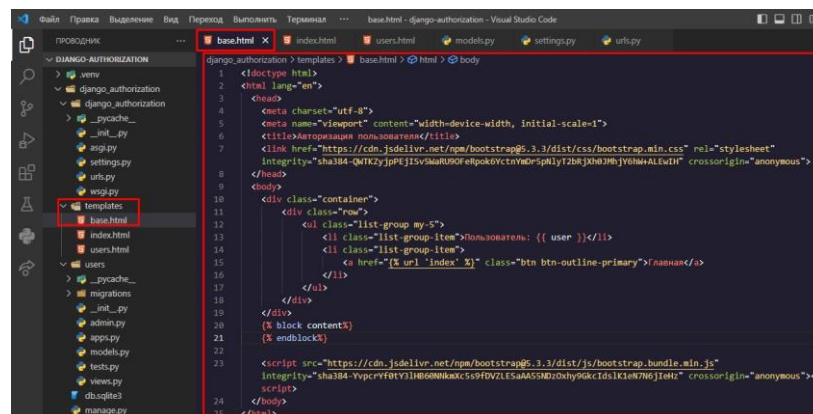


Рисунок 12 - Файл base.html

Изменим также файл index.html, будем расширяться от базового шаблона и добавим любую информацию для страницы, чтобы она не была слишком пустой (рисунок 13):

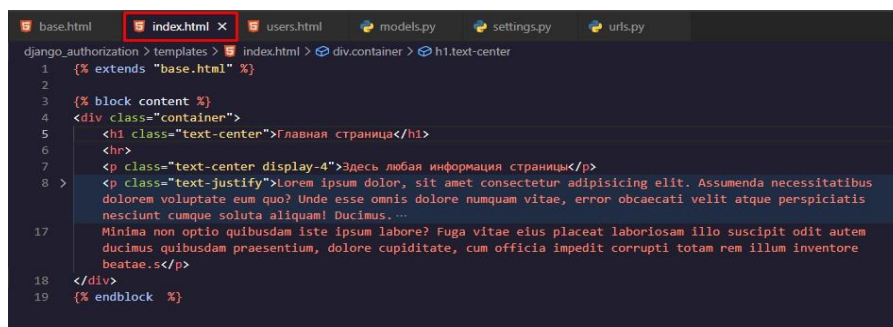
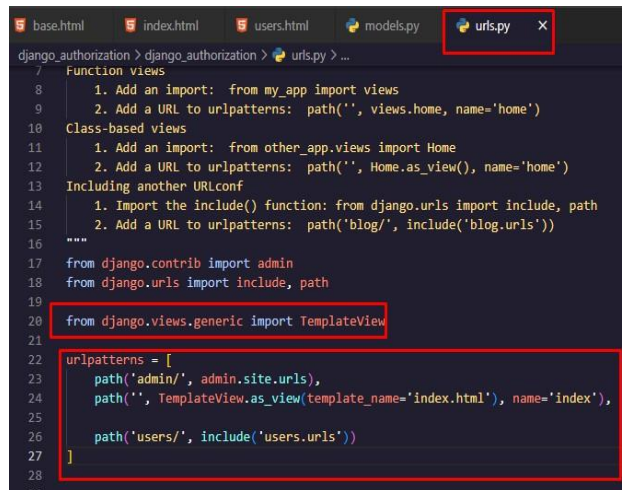


Рисунок 13 - Файл index.html

После проделанных действий, необходимо добавить маршрут для index.html. Раньше, мы делали это через отдельную вью-функцию, где просто возвращали render страницы. Поступим более профессионально и будем использовать TemplateView.

TemplateView — это класс, который служит для обработки шаблонов и отправки результата пользователю (рисунок 14).



```

django_authorization > django_authorization > urls.py > ...
7 Function views
8     1. Add an import: from my_app import views
9     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16
17 from django.contrib import admin
18 from django.urls import include, path
19
20 from django.views.generic import TemplateView
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', TemplateView.as_view(template_name='index.html'), name='index'),
25
26     path('users/', include('users.urls'))
27 ]
28
29

```

Рисунок 14 - Главный файл urls.py

Дополнительно создаем файл **urls.py** внутри приложения **users** (рисунок 15):

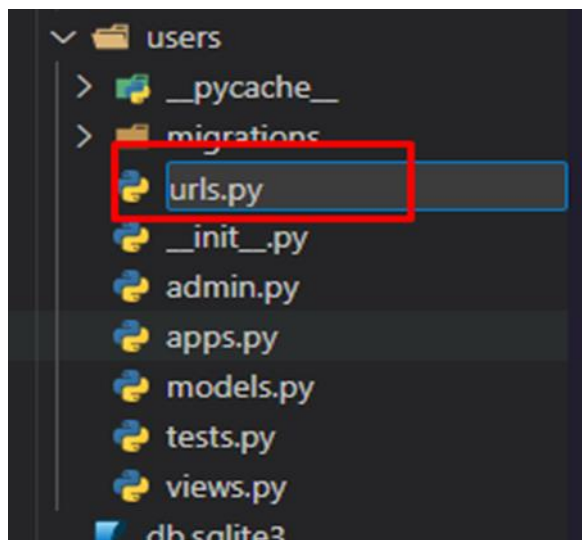


Рисунок 15 - Создание файла urls.py в приложении users

Сделаем так, чтобы мы могли просмотреть все маршруты, которые представляет Django для авторизации. Воспользуемся «include».

Функция **include()** в Django позволяет определить вложенные маршруты или подмаршруты для некоторого маршрута.

Пропишем следующий код для файла urls.py приложения users (рисунок 16):

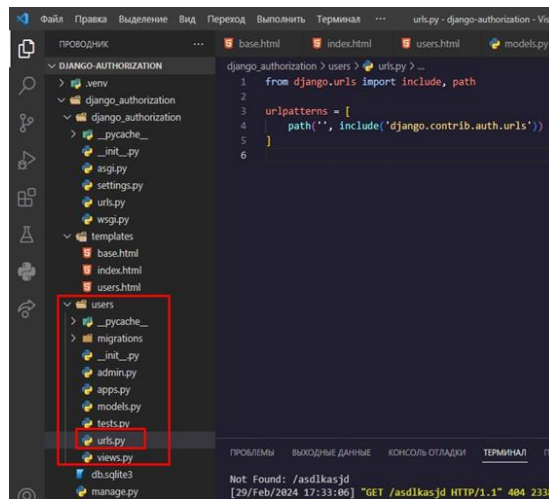


Рисунок 16 - Файл urls.py в приложении users

Если мы перейдем в адресной строке по пути users/ (который мы указали в главном urls.py файле), то можно заметить, что Django предлагает нам много различных путей (авторизоваться, выйти с аккаунта, смена пароля, подтверждение смены пароля, сброс пароля, подтверждение сброса пароля, сброс, подтверждение сброса и многое другое) (рисунок 17).

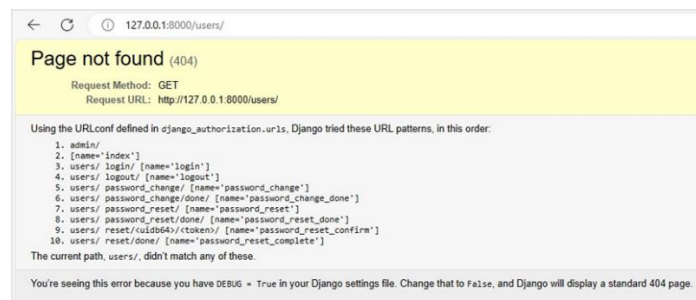


Рисунок 17 - Маршруты, предоставляемые для работы с авторизацией в Django

Далее, посмотрим, какие представления отвечают за отображение данных страниц. Для этого, можно зажать Control (CTRL) и нажать на **urls.** (где путь 'django.contrib.auth.urls') (рисунок 18):

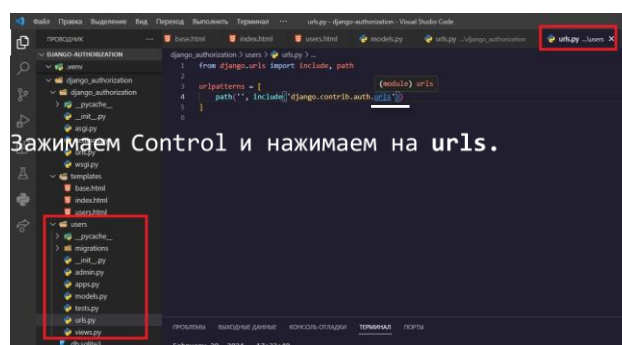


Рисунок 18 - Совет, как смотреть исходный код функций в Visual Studio Code

При переходе в данный файл, мы видим данные представления. Здесь видны урлы (urls), и у каждого url'a есть свое представление, которое обрабатывает запрос. Здесь отсутствует обработка регистрации пользователей, но мы это легко исправим, создав свою (рисунок 19).

```
from django.contrib.auth import views
from django.urls import path

urlpatterns = [
    path("login/", views.LoginView.as_view(), name="login"),
    path("logout/", views.LogoutView.as_view(), name="logout"),
    path(
        "password_change/", views.PasswordChangeView.as_view(), name="password_change"
    ),
    path(
        "password_change/done/",
        views.PasswordChangeDoneView.as_view(),
        name="password_change_done",
    ),
    path("password_reset/", views.PasswordResetView.as_view(), name="password_reset"),
    path(
        "password_reset/done/",
        views.PasswordResetDoneView.as_view(),
        name="password_reset_done",
    ),
    path(
        "reset/<uidb64>/<token>/",
        views.PasswordResetConfirmView.as_view(),
        name="password_reset_confirm",
    ),
    path(
        "reset/done/",
        views.PasswordResetCompleteView.as_view(),
        name="password_reset_complete",
    ),
]
```

Рисунок 19. Файл urls.py, отвечающий за авторизацию, находящийся внутри Django (исходники фреймворка)

Изменим немного внешнее оформление страницы base.html, добавим стили Bootstrap (рисунок 20):

```
<div class="container">
  <div class="row">
    <ul class="list-inline my-5">
      <li class="list-inline-item">Пользователь: {{ user }}</li>
      <li class="list-inline-item">
        <a href="{% url 'index' %}" class="btn btn-outline-primary">Главная</a>
      </li>
      <li class="list-inline-item">
        <a href="{% url 'register' %}" class="btn btn-outline-primary">Регистрация</a>
      </li>
    </ul>
  </div>
</div>
```

Рисунок 20 - Изменение базового шаблона base.html

Создадим страницу для регистрации register.html в папке templates (рисунок 21):

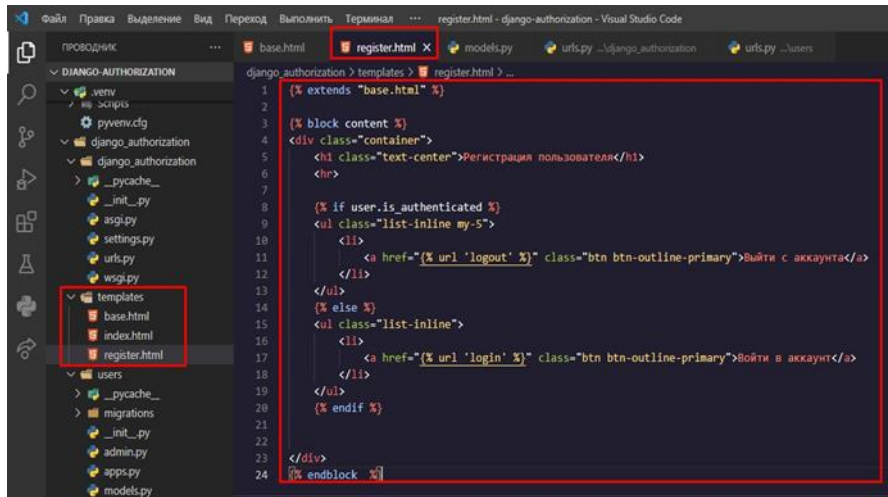


Рисунок 21 - Файл register.html

Перейдем на страницу регистрации и посмотрим, что будет если нажать на ссылку «Регистрация» (рисунок 22):

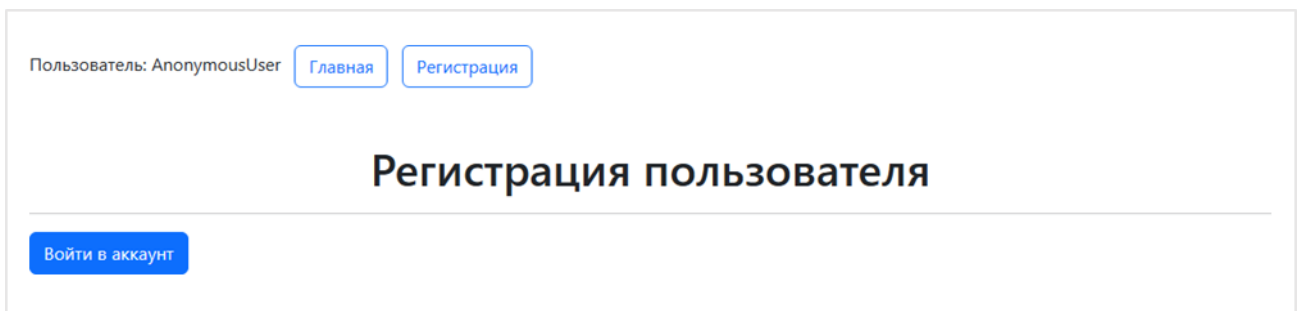


Рисунок 22 - Отображение страницы register.html

Попробуем нажать на ссылку «Войти в аккаунт» (рисунок 23):



Рисунок 23 - Ошибка TemplateDoesNotExist

Мы видим ошибку, возникающая потому, что у нас нет отображения для представления login.html (которая, в свою очередь находится в каталоге

registration). Мы должны создать все именно также! (рисунок 24):



Рисунок 24 - Создание представления для отображения login.html в папке registration

Если мы посмотрим на страницу, то ошибки больше нет (рисунок 25).

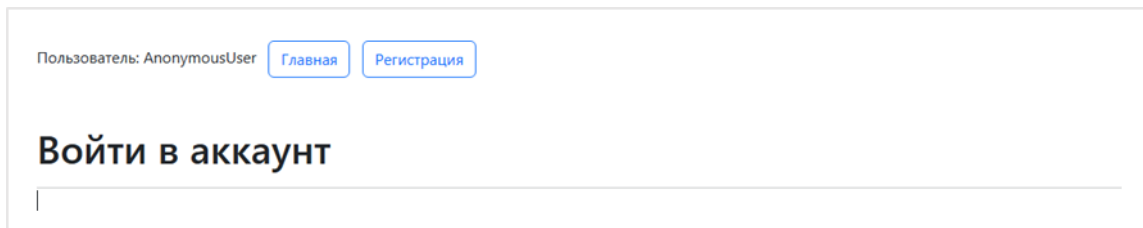


Рисунок 25 - Отображение страницы login.html

Если теперь, мы вернемся в файл urls.py, нажмем на urls (с зажатым CTRL) и перейдем в представление, которое отвечает за отображение авторизации (login), то мы увидим, что она работает за счет класса LoginView. Нажмем на него с зажатым CTRL (рисунок 26).

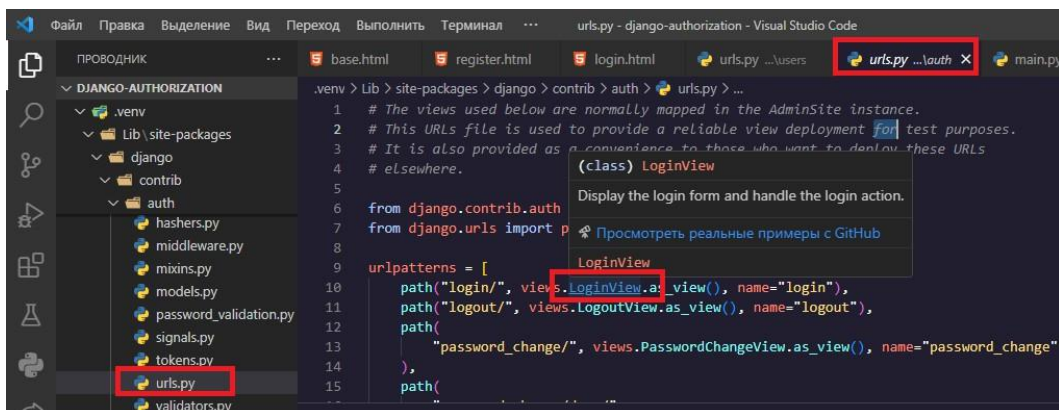


Рисунок 26 - Файл urls.py в исходниках Django

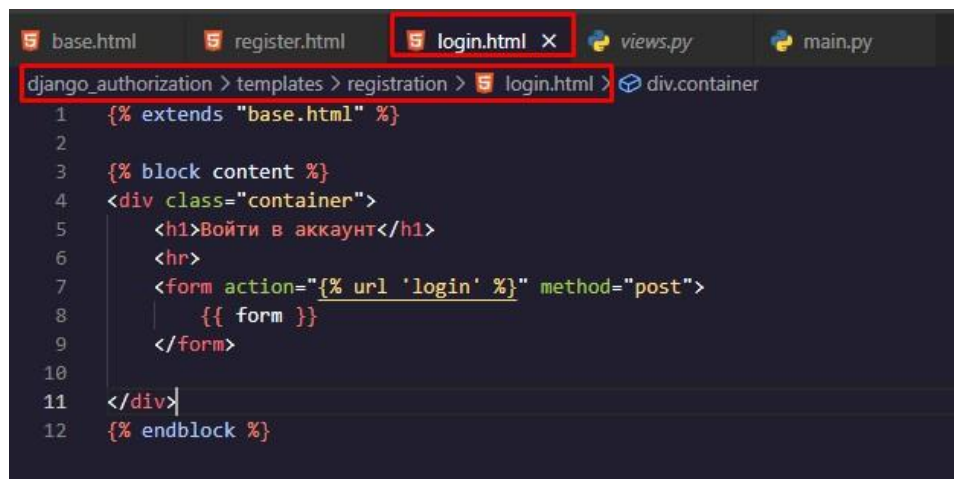
После нажатия на LoginView увидим следующий код (рисунок 27):

```
class LoginView(RedirectURLMixin, FormView):
    """
    Display the login form and handle the login action
    """

    form_class = AuthenticationForm
    authentication_form = None
    template_name = "registration/login.html"
    redirect_authenticated_user = False
    extra_context = None
```

Рисунок 27 - LoginView в исходниках Django

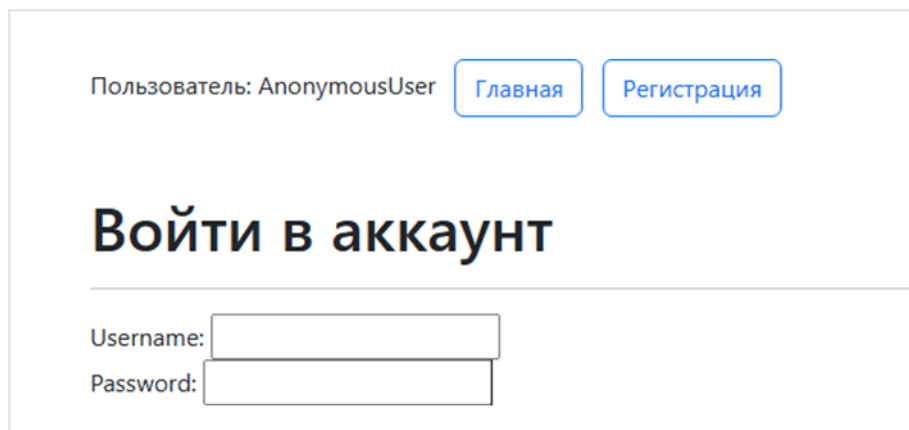
Мы увидим, что здесь используется форма AuthenticationForm. Мы также будем использовать для авторизации. Перейдем в файл login.html и добавим в него следующий код, чтобы отобразить полностью форму авторизации (рисунок 28).



```
1 {% extends "base.html" %}
2
3 {% block content %}
4 <div class="container">
5     <h1>Войти в аккаунт</h1>
6     <hr>
7     <form action="{% url 'login' %}" method="post">
8         {{ form }}
9     </form>
10
11 </div>
12 {% endblock %}
```

Рисунок 28 - Файл login.html

Перейдем на страницу авторизации и увидим результат (рисунок 29):



Пользователь: AnonymousUser [Главная](#) [Регистрация](#)

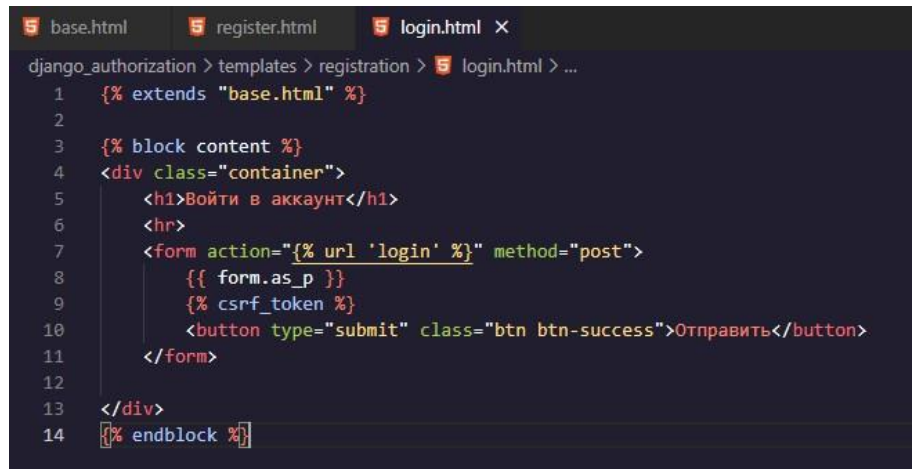
Войти в аккаунт

Username:

Password:

Рисунок 29 - Страница авторизации

Нужно добавить CSRF_TOKEN, ведь мы работаем с пост-запросами (post) и кнопку «Отправить» (рисунки 30-31).



```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5      <h1>Войти в аккаунт</h1>
6      <hr>
7      <form action="{% url 'login' %}" method="post">
8          {{ form.as_p }}
9          {% csrf_token %}
10         <button type="submit" class="btn btn-success">Отправить</button>
11     </form>
12
13 </div>
14 {% endblock %}

```

Рисунок 30 - Добавление csrf_token'а и кнопки «Отправить»

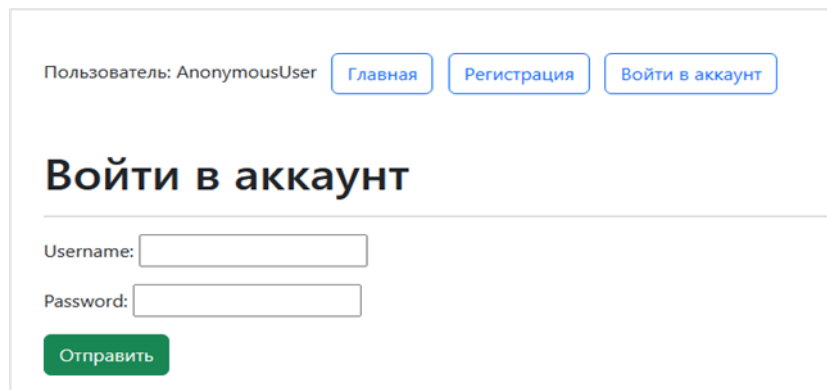
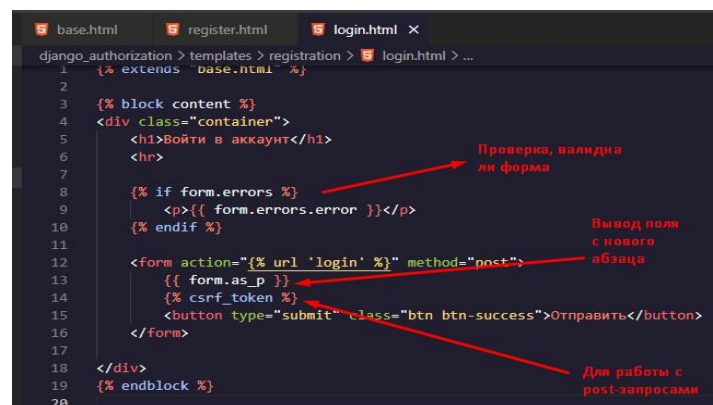


Рисунок 31 - Страница авторизации с кнопкой

Но, если мы заполним поля, и они будут некорректные, ничего не произойдет. Нужно обрабатывать ошибку, если данные будут не валидные.

Для этого, добавим условие if, и будем проверять ошибка. А также, будем выводить поля формы с нового абзаца, для этого используем {{ form.as_p }} (рисунок 32):



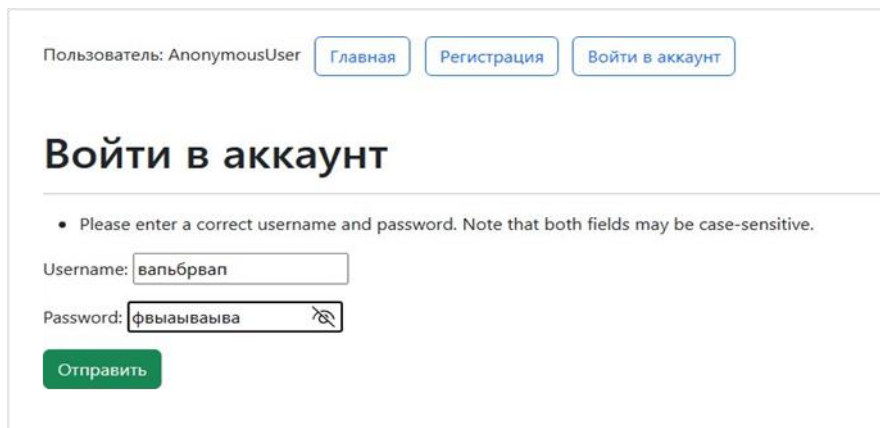
```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5      <h1>Войти в аккаунт</h1>
6      <hr>
7
8      {% if form.errors %}
9          <p>{{ form.errors.error }}</p>
10      {% endif %}
11
12      <form action="{% url 'login' %}" method="post">
13          {{ form.as_p }}
14          {% csrf_token %}
15          <button type="submit" class="btn btn-success">Отправить</button>
16      </form>
17
18 </div>
19
20 {% endblock %}

```

Рисунок 32 - Редактирование кода для файла login.html

Попробуем ввести любые данные для авторизации (рисунок 33):



Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Войти в аккаунт

- Please enter a correct username and password. Note that both fields may be case-sensitive.

Username:

Password:

[Отправить](#)

Рисунок 33 - Ввод невалидных данных для авторизации

Текст выводится на английском, это происходит из-за того, что в настройках нашего проекта (settings.py) указан языковой код 'en-us' (код языка для английского языка США), необходимо указать код для русского языка. Код русского языка – ru (Согласно – ISO 639). Исправим файл settings.py следующим образом (рисунок 34):

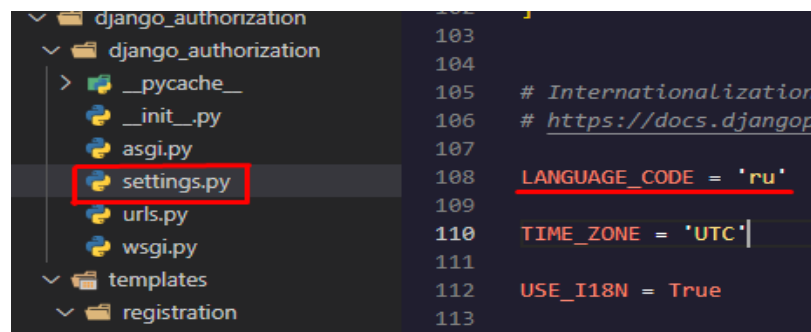
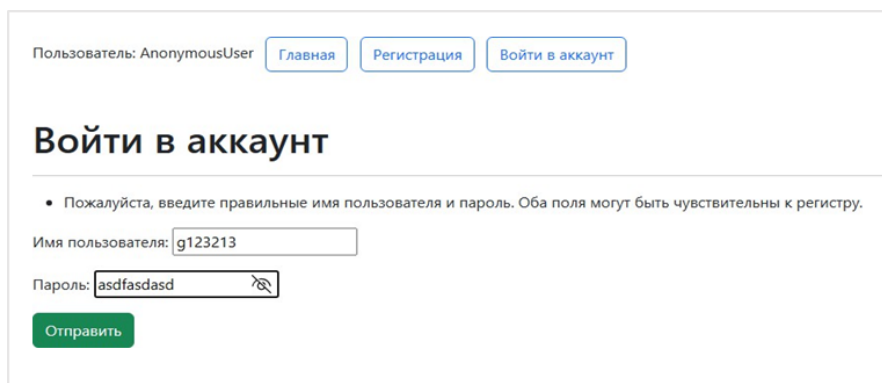


Рисунок 34 - Исправление языка в файле settings.py

После внесения изменений, язык будет изменен на русский (рисунок 35).



Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Войти в аккаунт

- Пожалуйста, введите правильные имя пользователя и пароль. Оба поля могут быть чувствительны к регистру.

Имя пользователя:

Пароль:

[Отправить](#)

Рисунок 35 - Отображение ошибок при авторизации на русском языке

Как вы помните, мы создавали суперпользователя. Попробуем авторизоваться под суперпользователем.

Вводим данные суперпользователя и нажимаем «Отправить» (рисунок 36):

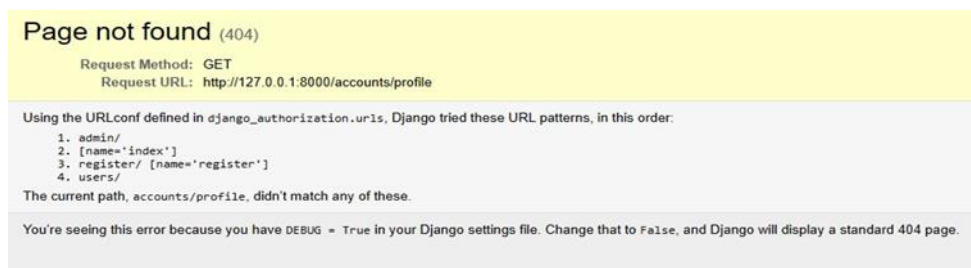


Рисунок 36 - Ошибка 404 (Page not found)

Происходит это из-за того, что нет данного пути, его можно как создать, так и ПЕРЕОПРЕДЕЛИТЬ. По-умному, нет смысла создавать новое представление просто для отправки, давайте же переопределим тот адрес, по которому нас будет направлять после авторизации. Точно также поступим и для случая, если пользователь вышел с аккаунта. Добавим LOGIN/LOGOUT REDIRECT URL'ы в файле настроек settings.py (рисунок 37):

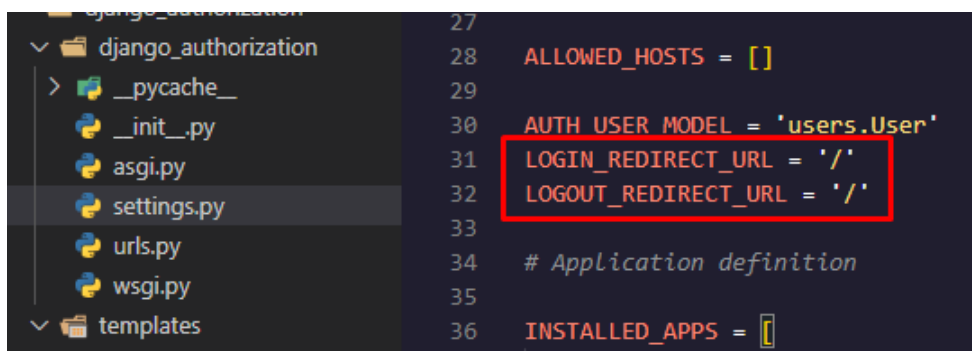


Рисунок 37- Добавление LOGIN_REDIRECT_URL и LOGOUT_REDIRECT_URL в файле settings.py

Если указать '/', переадресация будет на главную страницу.

Тот факт, что logout работает GET-запросом, был (вероятно) нарушением протокола HTTP и он позволял определенным сценариям выходить из аккаунта пользователей без их желания. Поэтому, если необходимо выходить из аккаунта пользователя, то нужно прописывать мини-шаблон формы для корректной работы выхода из аккаунта.

Изменим файл base.html и добавим прямо в него выход из аккаунта только в том случае, если мы авторизованы.

И удалим вход/выход с аккаунта со страницы регистрации (рисунок 38).

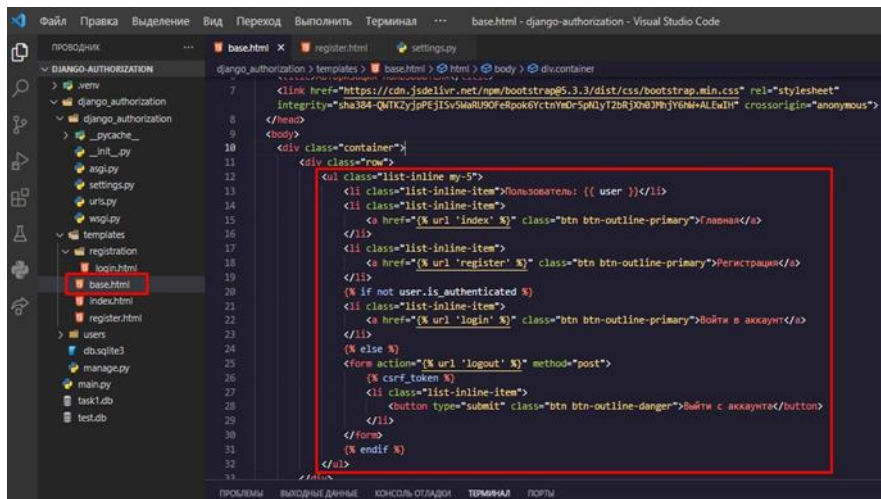


Рисунок 38 - Исправление кода для базового шаблона base.html

Изменим главный файл urls.py. Добавим маршрут для регистрации (рисунок 39):

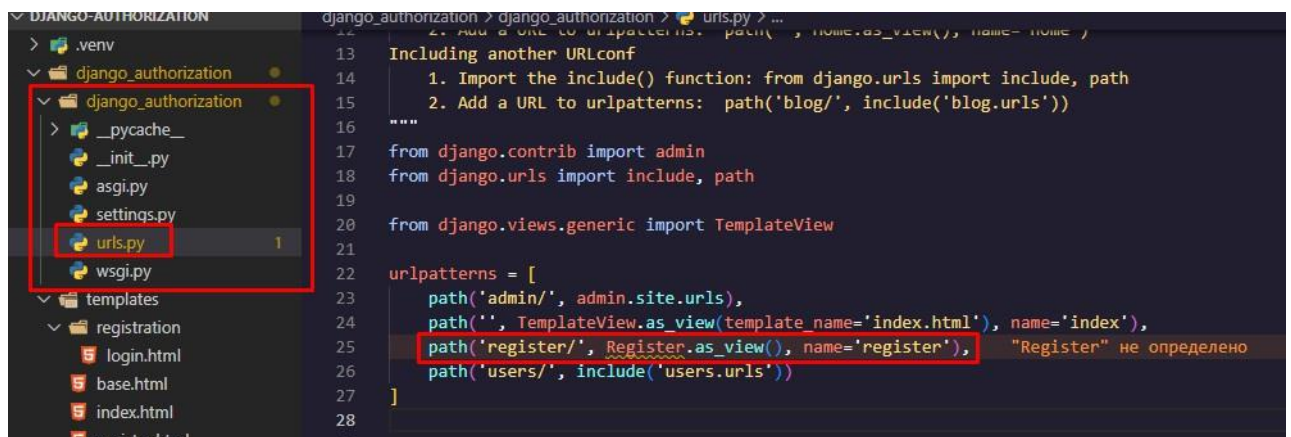


Рисунок 39 - Добавление маршрута для регистрации

Добавим выюшку для регистрации, в которой укажем переменную `template_name` и передадим ей путь до представления регистрации (`registration/register.html`). Укажем функцию **get()** для обработки запроса с методом GET, создадим в нем переменную для контекста и передадим форму для регистрации, которую импортировали из `django.contrib.auth.forms`.

Функция **get()** отвечает за обработку запроса с методом **GET**.

Аналогично, функция **post()** отвечает за обработку запроса с методом **POST** (рисунок 40).

```

django_authorization > users > views.py > Register > get
1  from django.shortcuts import render
2  from django.views import View
3  from django.contrib.auth.forms import UserCreationForm
4
5  class Register(View):
6
7      template_name = 'registration/register.html'
8
9      def get(self, request):
10         context = {
11             'form': UserCreationForm()
12         }
13         return render(request, self.template_name, context)

```

Рисунок 40 - Код файла views.py в django-приложении users

Чтобы маршрут, который мы указали в urls.py работал корректно, необходимо импортировать в данном файле класс Register, который мы только что создали (рисунок 41).

```

from users.views import Register

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', TemplateView.as_view(template_name='index.html'), name='index'),
    path('register/', Register.as_view(), name='register'),
    path('users/', include('users.urls'))
]

```

Рисунок 41 - Импорт класса для регистрации из файла views.py приложения users

Необходимо перенести файл register.html в каталог registration (рисунок 42).

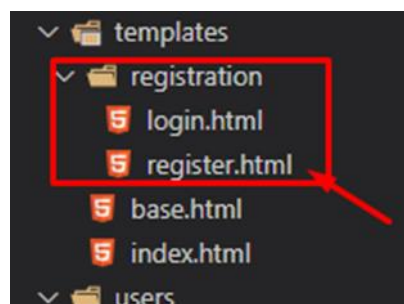


Рисунок 42 - Папка registration, содержащая в себе login.html и register.html

Перейдем на сайт, и убедимся, что страница регистрации отображается корректно (рисунок 43):

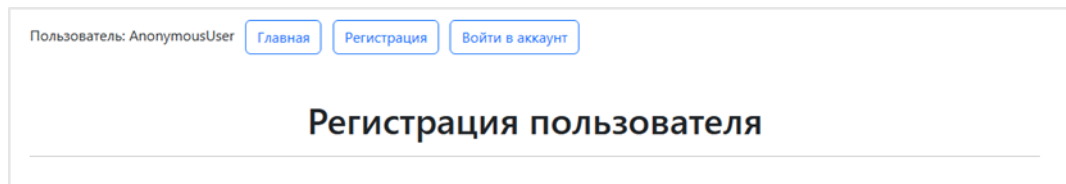


Рисунок 43 - Страница регистрации

Добавим в register.html следующий код (рисунок 44):

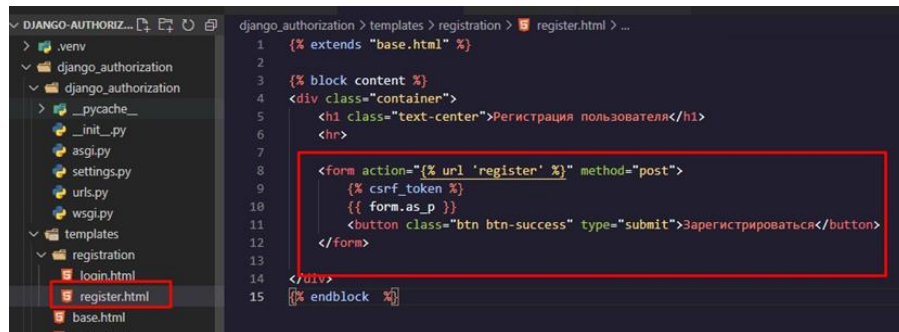


Рисунок 44 - Редактирование кода в файле регистрации

Создадим файл forms.py (рисунок 45):

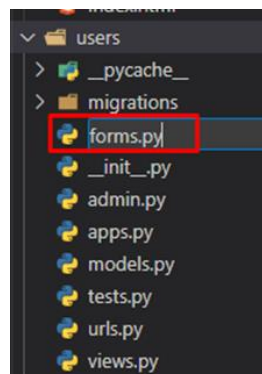


Рисунок 45 - Создание файла для форм (forms.py)

Добавим в файл форм следующий код (рисунок 46):

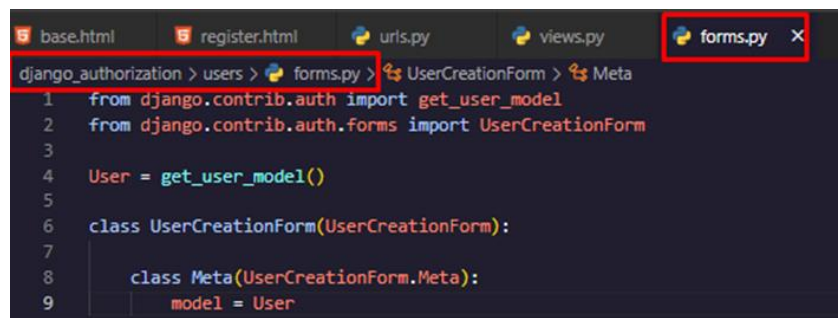


Рисунок 46 - Код файла forms.py

И добавим функцию get для регистрации, в которой будем проверять валидность формы, пришедшая от пользователя, если форма валидна, то берем данные из полей username и password1 и авторизовываем пользователя (рисунок 47).

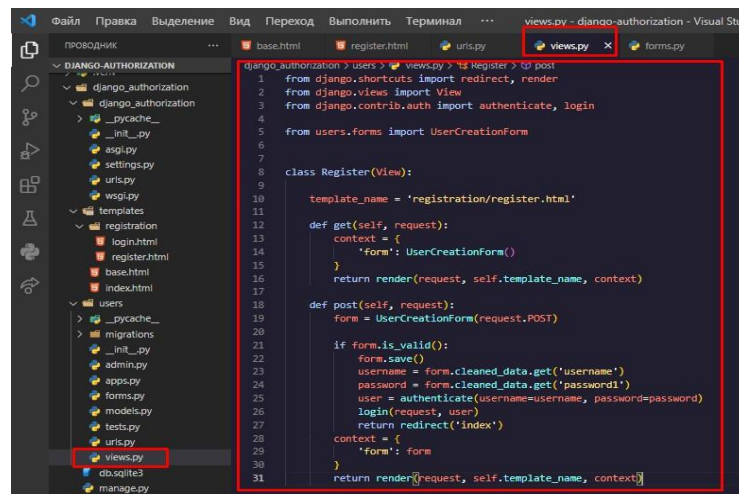


Рисунок 47 - Редактирование файла views.py (добавление функции post)

Регистрируем класс пользователя в файле admin.py, чтобы в административной панели Django мы могли просматривать пользователей (рисунок 48):

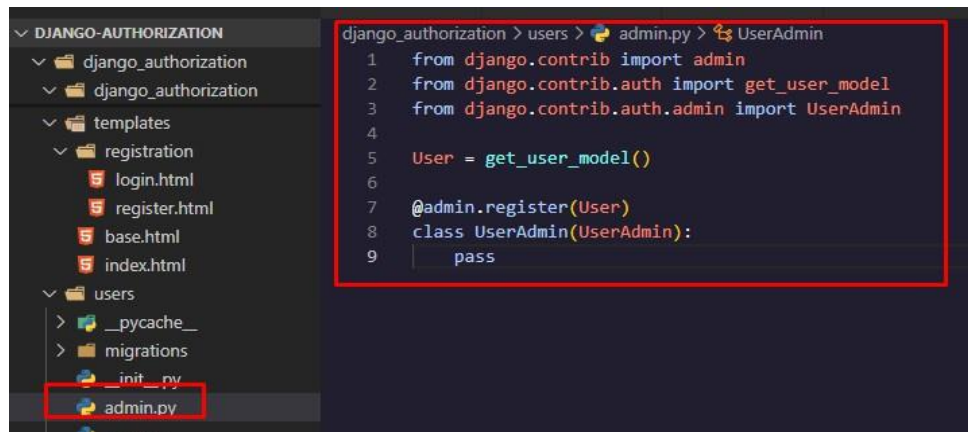


Рисунок 48 - Регистрация класса User и создание класса UserAdmin в admin.py

Зайдем после всех действий на сайт, и увидим следующую картину (рисунок 49):

Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Регистрация пользователя

Имя пользователя: Обязательное поле. Не более 150 символов. Только буквы, цифры и символы @/./+/-/_.

Пароль:

- Пароль не должен быть слишком похож на другую вашу личную информацию.
- Ваш пароль должен содержать как минимум 8 символов.
- Пароль не должен быть слишком простым и распространенным.
- Пароль не может состоять только из цифр.

Подтверждение пароля: Для подтверждения введите, пожалуйста, пароль ещё раз.

[Зарегистрироваться](#)

Рисунок 49 - Файл регистрации с отображением всех ошибок/рекомендаций

Выглядит как-то скучно, да и все ошибки/рекомендации по заполнению полей сразу отображаются. Для более детальной настройки, изменим файл register.html, и применим стили Bootstrap: (рисунок 50):

```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5    <h1 class="text-center">Регистрация пользователя</h1>
6    <div>
7      <form action="{% url 'register' %}" method="post">
8        <div class="row justify-content-center">
9          <div class="col-6">
10             <div class="form-group">
11               <label for="id_username" class="mb-2">Имя пользователя</label>
12               <input type="text" name="username" id="id_username" class="form-control mb-3" placeholder="Введите имя пользователя">
13             </div>
14             <label for="id_password1" class="mb-2">Пароль</label>
15             <input type="password" name="password1" id="id_password1" class="form-control mb-3" placeholder="Введите имя пользователя">
16             <label for="id_password2" class="mb-2">Подтверждение пароля</label>
17             <input type="password" name="password2" id="id_password2" class="form-control mb-3" placeholder="Введите имя пользователя">
18             <div class="text-center">
19               <button class="btn text-center btn-success" type="submit">Зарегистрироваться</button>
20             </div>
21           </div>
22         </div>
23       </form>
24     </div>
25   </div>
26 </div>
27 </div>
28 </div>
29 </div>
30 </div>
31 </div>
32 </div>

```

Рисунок 50 - Код файла register.html

Хоть и выглядит все страшно, но тут все предельно просто.

В конечном итоге, Вы получите регистрацию и авторизацию пользователя, а также отображение зарегистрированных пользователей в административной панели Django.

Реализуем довольно-таки сложную на первый взгляд логику, для отображения ошибок полей в форме. Делается это через form.errors.

Основная суть обработки ошибок при валидации формы: мы прописываем условие прямо внутри атрибута class (HTML), и здесь проверяется, есть ли ошибка в поле «Имя пользователя». is-invalid применяется к полю, содержащего класс form-control в случае возникновения какой-либо ошибки (Если ошибка есть, контур поля будет светиться красным цветом)

Далее у нас идет условие для отображения непосредственно ошибки (Внутри Django данная логика уже реализована, и нам нужно лишь правильно это написать; Django за нас будет отображать определенные ошибки)В условии мы пишем, что если в поле имени пользователя содержится ошибка, то будет выводиться первая ошибка (т.е. та, в которой пользователь ошибся; т.е. данное условие проходится по всем ошибкам в списке ошибок (см. скриншот СПИСОК ВСЕХ ОШИБОК выше))

в блоке `invalid-feedback`, которое будет отображаться ПОД вводимым полем (рисунок 51):

```
<label for="id_username" class="mb-2">Имя пользователя</label>
<input type="text" name="username" id="id_username" class="form-control mb-3 {% if form.username.errors %} is-invalid {% endif %}" placeholder="Введите имя пользователя">
{% if form.username.errors %}
  <div class="invalid-feedback">
    {{ form.username.errors.0 }}
  </div>
{% endif %}
```

Рисунок 51 - Отображение ошибок для поля username

Примеры отображения ошибок (рисунки 52-55):

Имя пользователя

gaziikk

Пользователь с таким именем уже существует.

Рисунок 52 - Ошибка «Пользователь с таким именем уже существует»

Имя пользователя

Введите имя пользователя

Обязательное поле.

Рисунок 53 - Ошибка «Обязательное поле»

Имя пользователя

текст больше 150 символовтекст больше 150 символовтекст больш

Убедитесь, что это значение содержит не более 150 символов (сейчас 975).

Рисунок 54 - Ошибка «Убедитесь, что это значение содержит не более 150 символов»

Имя пользователя

пользователь~

Введите правильное имя пользователя. Оно может содержать только буквы, цифры и знаки @./+/-/_.

Рисунок 55 - Ошибка «Введите правильное имя пользователя. Оно может содержать только буквы, цифры и знаки»

4.2 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте систему регистрации пользователя с использованием встроенной модели `User` Django. Реализуйте форму регистрации с полями «Имя пользователя», «Электронная почта» и «Пароль». Обеспечьте сохранение данных пользователя в базе данных и последующий переход на страницу успешной регистрации или главную страницу сайта.

Решение задачи:

```
1 forms.py (создаём форму регистрации)
from django import forms
class RegisterForm(forms.Form):
    username = forms.CharField(max_length=150)
    email = forms.EmailField()
    password = forms.CharField(widget=forms.PasswordInput)
2 views.py (логика регистрации)
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from .forms import RegisterForm
def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']
            # создание пользователя
            user = User.objects.create_user(
                username=username,
                email=email,
                password=password
            )
            user.save()
            return redirect('register_success')
        else:
            form = RegisterForm()
            return render(request, 'posts/register.html', {'form': form})
def register_success(request):
    return render(request, 'posts/register_success.html')
3 urls.py (маршруты)
from django.urls import path
from . import views
urlpatterns = [
    path('register/', views.register, name='register'),
    path('register/success/', views.register_success, name='register_success'),
```

```

]
4 templates/posts/register.html
<!DOCTYPE html>
<html>
<head>
  <title>Регистрация</title>
</head>
<body>
<h2>Регистрация пользователя</h2>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Зарегистрироваться</button>
</form>
</body>
</html>
5 templates/posts/register_success.html
<!DOCTYPE html>
<html>
<head>
  <title>Успех</title>
</head>
<body>
<h2>Регистрация прошла успешно!</h2>
<a href="/">На главную</a>
</body>
</html>

```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте `library_site` и приложении `users` создайте систему регистрации пользователя с полями «Имя пользователя», «Электронная почта» и «Пароль». Реализуйте сохранение данных пользователя с использованием встроенной модели `User Django`.

2 В ранее созданном проекте `student_portal` и приложении `accounts` создайте форму входа пользователя в систему. Реализуйте проверку логина и пароля с использованием функции `authenticate()` и авторизацию через `login()`.

3 В ранее созданном проекте `movie_portal` и приложении `profiles` создайте страницу регистрации нового пользователя с использованием Bootstrap-классов `form-control` и `btn btn-primary`.

4 В ранее созданном проекте `music_library` и приложении `accounts` создайте систему выхода пользователя из аккаунта с использованием функции `logout()`. После выхода перенаправьте пользователя на главную страницу сайта.

5 В ранее созданном проекте `online_shop` и приложении `users` создайте страницу авторизации пользователя с полями «Логин» и «Пароль». При успешном входе выводите приветствие с именем пользователя.

6 В ранее созданном проекте `news_portal` и приложении `accounts` создайте страницу регистрации и реализуйте проверку совпадения пароля и подтверждения пароля.

7 В ранее созданном проекте `tourism_portal` и приложении `tourists` создайте закрытую страницу «Личный кабинет», доступную только авторизованным пользователям с использованием декоратора `login_required`.

8 В ранее созданном проекте `sport_club_site` и приложении `members` создайте форму регистрации пользователя с полями «Имя пользователя», «Пароль» и «Подтверждение пароля».

9 В ранее созданном проекте `school_portal` и приложении `teachers` создайте страницу входа преподавателя в систему. Используйте Bootstrap-класс `btn btn-success` для кнопки отправки формы.

10 В ранее созданном проекте `hospital_system` и приложении `patients` создайте систему регистрации пациента с последующей автоматической авторизацией после успешной регистрации.

11 В ранее созданном проекте `education_platform` и приложении `accounts` создайте страницу изменения пароля пользователя с использованием встроенных средств Django Authentication System.

12 В ранее созданном проекте `restaurant_site` и приложении `staff` создайте систему авторизации сотрудников ресторана. После успешного входа отображайте имя текущего пользователя в шаблоне.

13 В ранее созданном проекте `car_catalog` и приложении `drivers` создайте страницу регистрации и страницу входа пользователя. Реализуйте перенаправление неавторизованных пользователей на страницу входа при попытке открыть защищённую страницу.

14 В ранее созданном проекте `portfolio_site` и приложении `main` создайте систему регистрации пользователя с использованием встроенной модели `User` Django. Реализуйте форму регистрации с полями «Имя пользователя», «Электронная почта» и «Пароль». Обеспечьте сохранение данных пользователя в базе данных и отображение списка зарегистрированных пользователей на отдельной странице.

15 В ранее созданном проекте `home_site` и приложении `home` создайте систему регистрации пользователя с использованием встроенной модели `User` Django. Реализуйте форму регистрации с полями «Имя пользователя», «Электронная почта» и «Пароль». Обеспечьте обработку введённых данных во `views.py`, сохранение пользователей в базе данных и вывод списка зарегистрированных пользователей на отдельной странице.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Для чего используется встроенная модель User в Django в языке программирования Python?

7.2 Что понимается под аутентификацией пользователей в Django в языке программирования Python?

7.3 Для чего применяется функция `authenticate()` в языке программирования Python?

7.4 Для чего используется объект `request.user` в Django в языке программирования Python?

7.5 Как подключить Bootstrap для оформления форм авторизации в языке программирования Python?

Список используемых источников

1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.